

Modül 12: Koleksiyonlar

Dört biçim, her birinin birkaç uygulaması. Doğru seçmek becerinin çoğu, kalanı da hangi üç dört tuzağın var olduğunu bilmek.

O tuzaklardan ikisi, zaten iki kez karşılaştığınız aynı kural. Modül 06 bir `HashSet` içinde nesne kaybetti; Modül 08 record'daki değiştirilebilir bileşen konusunda uyardı. İkisi de bu modülün kuralıydı, farklı açılardan görülmüş.

Neler Öğreneceksiniz

- `List`, `Set`, `Map`, `Deque` ve hangi uygulamaya uzanacağınız
- Sıralı koleksiyonlar ve kapattıkları otuz yıllık boşluk
- `List.of` ve arkadaşları, insanları şaşırtan dört davranışla
- Yineleme sırasında silmenin neden fırlattığı ve dört doğru alternatif
- Anahtarlar hakkındaki kural, nihayet açıkça

Başka Bir Dilden Geliyorsanız

	Python	Java
Sıralı dizi	<code>list</code>	<code>ArrayList</code>
Benzersiz	<code>set</code>	<code>HashSet</code> , sıra için <code>LinkedHashSet</code>
Anahtardan değere	<code>dict</code> , ekleme sıralı	<code>HashMap</code> sırasız, <code>LinkedHashMap</code> sıralı
Sıralı	istendiğinde <code>sorted()</code>	<code>TreeMap</code> , <code>TreeSet</code> sıralı tutar
Değişmez sabit	<code>tuple</code> , <code>frozenset</code>	<code>List.of</code> , <code>Set.of</code> , <code>Map.of</code>
Yığın / kuyruk	<code>list</code> , <code>deque</code>	<code>ArrayDeque</code>

Python geliştiricilerini yakalayan satır: `HashMap` ekleme sırasını korumaz. Python sözlükleri 3.7'den beri koruyor. Sıra ihtiyacınız varsa `LinkedHashMap` kullanarak bunu söyleyin.

Ders

Seçmek

`ChoosingACollection.java` dosyasından:

İhtiyaç	Kullanın
indeks ya da tekrar	ArrayList
benzersizlik, sıra önemsiz	HashSet
benzersizlik, kararlı çıktı	LinkedHashSet
sıralı	TreeSet / TreeMap
anahtardan değere	HashMap ya da LinkedHashMap
yiğın ya da kuyruk	ArrayDeque

ArrayList ile LinkedList karşılaştırması: ArrayList kullanın. Bağlı liste *bilinen bir konumda* $O(1)$ ekleme verir, ama o konumu bulmak $O(n)$ 'dir ve her eleman fazladan bir nesne artı iki işaretçiye mal olur. Ardışık bellek hızlıdır, işaretçi takibi değil, dolayısıyla büyük O tablosu kaybetmesi gerektiğini söylediği yerlerde bile ArrayList pratikte kazanır.

java.util.Stack asla kullanmayın. Vector sınıfını genişletir, 1998'de terk edilmiş bir iş parçacığı modeli için her metodu senkronize eder ve alttan üste yineler, ki bu bir yiğın için terstir. ArrayDeque hem yiğın hem kuyruk için cevaptır.

Bir HashSet yineleme sırası bir söz değildir. Testlerinizde kararlı görünmesi hiçbir şey ifade etmez; JDK sürümleri arasında değişebilir.

Hemen öğrenmeye değer üç Map metodu:

```
map.getOrDefault(key, 0)
map.computeIfAbsent(key, k -> new ArrayList<>()).add(value) // map of lists
map.merge(word, 1, Integer::sum) // counting
```

Sıralı koleksiyonlar

Java 21'de, 1998'den beri açık duran bir boşluğu kapatmak için eklendi. İlk elemanı almak tipe göre üç farklı ifadeydi ve bir LinkedHashSet kümesinin **son** elemanını almak tamamını yinelemek demekti.

[SequencedCollections.java](#) dosyasından:

```
getFirst: a
getLast: c
reversed: [c, b, a]
after add, the view shows: [d, c, b, a]
```

`reversed()` bir **görünümdür**, kopya değil. Oluşturması bedavadır ve özgün üzerindeki sonraki değişiklikleri yansıtır.

SequencedMap ayrıca `firstEntry`, `lastEntry`, `putFirst` ve `putLast` ekler:

```
firstEntry: x=1
putFirst: {w=0, x=1, y=2, z=3}
```

Arayüz	Uygulayanlar
SequencedCollection	List , Deque , LinkedHashSet , SortedSet
SequencedSet	LinkedHashSet , TreeSet
SequencedMap	LinkedHashMap , TreeMap

HashSet ve HashMap bilerek sıralı **değildir**. Tanımlı bir sıraları yoktur, dolayısıyla derleyici artık soruyu, yinelemenize ve umut etmenize izin vermek yerine reddeder.

Değişmez fabrikalar

`ImmutableFactories.java` dosyasından dört sürpriz, hepsi bilinçli:

1. **Gerçekten değişmez.** Değiştiren her metot fırlatır. "Gelenek olarak değiştirilemez" değil.
2. **null reddedilir.** `List.of("a", null)` bir `NullPointerException` fırlatır. Eski kodu `List.of` kullanacak şekilde dönüştürmek, var olduğunu bilmediğiniz null değerleri ortaya çıkarabilir.
3. **Tekrarlar hatadır.** `Set.of(1, 1)` çağrısı `duplicate element: 1` fırlatır, `new HashSet<> (List.of(1, 1))` ise sessizce tek eleman verirdi. Bir sabit listede tekrar neredeyse her zaman yazım hatasıdır.
4. **Set.of yineleme sırası belirsizdir** ve bilerek değişir. Sıraya ihtiyacınız varsa `List.of` ya da `LinkedHashSet` kullanın.

Birbirine benzeyen ama olmayan üç şey:

	add	set
<code>List.of</code>	fırlatır	fırlatır
<code>Arrays.asList</code>	fırlatır	çalışır
<code>Collections.unmodifiableList</code>	fırlatır	fırlatır

`Arrays.asList` **sabit boyutludur, değişmez değil**, ve arkasındaki diziye yazar.

`Collections.unmodifiableList` değiştirilebilir bir listenin **görünümüdür**:

```
after mutating the source:
unmodifiable VIEW sees it: [x, y]
List.copyOf does not:      [x]
```

Genellikle istediğiniz `List.copyOf` tur, Modül 05 ve 08'in ona uzanmasının sebebi de bu.

Yineleme sırasında değiştirme

`MutationDuringIteration.java` fırlatır:

```
Exception in thread "main" java.util.ConcurrentModificationException
at java.base/java.util.ArrayList$Itr.checkForComodification
```

"Concurrent" yanıltıcı. Tek bir iş parçacığı var. Koleksiyonun başka bir iş parçacığından değil, devam eden bir yineleme ile eş zamanlı olarak değiştiği anlamına gelir.

`ArrayList` her yapısal değişiklikte artan bir `modCount` tutar. Yineleyici onu kaydeder ve her adımdan önce denetler, eleman atlamak ya da sonunu aşmak yerine hızlı başarısız olur. Belgelerde **en iyi çaba** olarak geçer, dolayısıyla asla istisnanın fırlatılmasına dayanan kod yazmayın.

Dört doğru alternatif:

```
numbers.removeIf(n -> n % 2 == 0);           // 1. usually this

var it = numbers.iterator();                  // 2. when a predicate is not enough
while (it.hasNext()) {
    if (it.next() % 2 == 0) it.remove();
}

// 3. collect, then remove after
// 4. build a new collection (Module 14)
```

Anahtarlar hakkındaki kural

`KeysMustNotChange.java` değiştirilebilir bir nesneyi haritaya koyup yeniden adlandırıyor:

```
after renaming the key object:
get(key):      null
containsKey:   false
size:          1
but iterating still finds it: [MutableKey(invoices)]
after remove(key), size is still: 1
```

Girdi, **eski** hash kodunun kovalarında duruyor. Arama yenisini hesaplıyor, farklı bir kovaya bakıyor ve hiçbir şey bulamıyor. Harita onu hâlâ sayıyor ve yineleme hâlâ veriyor, dolayısıyla veri kaybolmadı. Yalnızca aramayla ulaşılamaz, ki bir haritanın var olma sebebi tek şey budur. Ayrıca artık anahtarla silinemez.

Harita anahtarları ve küme elemanları, `equals` ve `hashCode` metotlarının kullandığı her alanda değişmez olmalıdır.

Bileşenleri de değişmez olan bir record bunu bedavaya karşılar. Record'lar ile haritaların bu kadar iyi anlaşmasının sebebi çoğunlukla budur, ve bu Modül 08'in uyarısının diğer taraftan tekrarıdır.

Çalıştırın

```
java modules/12-collections/examples/ChoosingACollection.java
java modules/12-collections/examples/SequencedCollections.java
java modules/12-collections/examples/ImmutableFactories.java
java modules/12-collections/examples/KeysMustNotChange.java

# Bilerek başarısız olur.
java modules/12-collections/examples/MutationDuringIteration.java

./scripts/verify-examples.sh modules/12-collections
```

Sık Yapılan Hatalar

`HashMap` ya da `HashSet` yinleme sırasına güvenmek. Tanımlı değildir ve değişir.

`for-each` döngüsü içinde silmek. `removeIf` kullanın.

Değiştirilebilir bir nesneyi anahtar olarak kullanmak. İlgili bir alan değiştiği anda girdi ulaşamaz olur.

`Arrays.asList` sonucunun değişmez olmasını beklemek. Sabit boyutludur ve `set` arkasındaki diziye yazar.

`Collections.unmodifiableList(internalList)` döndürüp güvenli sanmak. Bu bir görünümdür. Özgünü elinde tutan hâlâ çağırının gördüğünü değiştirebilir.

Ekleme $O(1)$ diye `LinkedList` seçmek. Ekleme noktasını bulmak öyle değil ve sabit çarpanlar kötü.

`java.util.Stack` kullanmak. Sebepsiz senkronize, ve ters yönde yineliyor.

Özet Çıkarımlar

- `ArrayList`, `HashMap`, `ArrayDeque` çoğu ihtiyacı karşılar. Başka yere belirtilmiş bir sebeple uzanın.
- Sıranın görülebildiği her yerde `LinkedHashSet` ya da `LinkedHashMap` kullanın, örneğin çıktıda ve testlerde.
- Sıralı koleksiyonlar `getFirst`, `getLast` ve `reversed()` verir, ve `reversed()` bedava bir görünümdür.
- `List.of` null'ı reddeder, küme tekrarlarını reddeder ve gerçekten değişmezdir. `Arrays.asList` ve `unmodifiableList` ikisi de değil.
- Döngü içinde silme değil `removeIf`.
- Anahtarlar, `equals` ya da `hashCode` metotlarının okuduğu hiçbir alanda değişmemelidir.

Ödev

[homework/README.md](#)

Bir kelime dizini kurun, her adımda doğru koleksiyonu seçin ve bilerek bir harita girdisini ulaşılamaz kılın.
Referans çözüm [solutions/12-collections/](#) altında.